

第3章 组件基础（上）

学习目标/Target

熟悉选项式API和组合式API，能够说出选项式API和组合式API的区别

掌握生命周期函数的使用方法，能够灵活运用生命周期函数在特定时间执行特定的操作

掌握注册组件的方法，能够运用全局注册或者局部注册的方式完成组件的注册

掌握引用组件的方法，能够在Vue项目中以标签的形式引用组件

学习目标/Target

掌握组件之间样式冲突问题的解决方法，能够运用<style>标签的scoped属性和深度选择器解决组件之间样式冲突的问题

掌握父组件向子组件传递数据的方法，能够使用props实现数据传递

掌握子组件向父组件传递数据的方法，能够使用自定义事件实现数据传递

掌握跨级组件之间的传递数据的方法，能够使用依赖注入实现数据共享

章节概述/ Summary

在学习完第2章的基础知识后，读者应该已经可以编写一些简单的组件了，但是这样的组件功能比较简单，无法满足实际项目开发中各种复杂的需求。为了能够更灵活地使用组件，读者还需要更深入地学习组件的相关知识。本书将用第3章和第4章共两章的篇幅详细讲解组件基础，本章为上半部分内容。

目录/Contents

3.1

选项式API和组合式API

3.2

生命周期函数

3.3

组件的注册和引用

3.4

解决组件之间的样式冲突

目录/Contents

3.5

父组件向子组件传递数据

3.6

子组件向父组件传递数据

3.7

跨级组件之间的数据传递

3.8

阶段案例——待办事项

3.1

选项式API和组合式API

3.1 选项式API和组合式API



- 熟悉选项式API和组合式API，能够说出选项式API和组合式API的区别

3.1 选项式API和组合式API

Vue 3支持选项式API和组合式API。其中，选项式API是从Vue 2开始使用的一种写法，而Vue 3新增了组合式API的写法。



选项式API

选项式API是一种通过包含多个选项的对象来描述组件逻辑的API，其常用的选项包括data、methods、computed、watch等。

组合式API

相比于选项式API，组合式API是将组件中的数据、方法、计算属性、侦听器等代码全部组合在一起，写在setup()函数中。

3.1 选项式API和组合式API

选项式API的语法格式如下。

```
<script>
export default {
  data() {
    return { // 定义数据 }
  },
  methods: { // 定义方法 },
  computed: { // 定义计算属性 },
  watch: { // 定义侦听器 }
}
</script>
```

3.1 选项式API和组合式API

组合式API的语法格式如下。

```
<script>
import { computed, watch } from 'vue'
export default {
  setup() {
    const 数据名 = 数据值
    const 方法名 = () => {}
    const 计算属性名 = computed(() => {})
    watch(侦听器的来源, 回调函数, 可选参数)
    return { 数据名, 方法名, 计算属性名 }
  }
}
</script>
```

3.1 选项式API和组合式API

Vue还提供了`setup`语法糖，用于简化组合式API的代码。使用`setup`语法糖时，`组合式API`的语法格式如下。

```
<script setup>
import { computed, watch } from 'vue'
// 定义数据
const 数据名 = 数据值
// 定义方法
const 方法名 = () => {}
// 定义计算属性
const 计算属性名 = computed(() => {})
// 定义侦听器
watch(侦听器的来源, 回调函数, 可选参数)
</script>
```

3.1 选项式API和组合式API

选项式API和组合式API的关系

Vue提供的选项式API和组合式API这两种写法可以覆盖大部分的应用场景，它们是同一底层系统所提供的两套不同的接口。选项式API是在组合式API的基础上实现的。

3.1 选项式API和组合式API

企业在开发大型项目时，随着业务复杂度的增加，代码量会不断增加。

如果使用选项式API，整个项目逻辑不易阅读和理解，而且查找对应功能的代码会存在一定难度。

如果使用组合式API，可以将项目的每个功能的数据、方法放到一起，这样不管项目的大小，都可以快速定位到功能区域的相关代码，便于阅读和维护。同时，组合式API可以通过函数来实现高效的逻辑复用，这种形式更加自由，需要开发者有较强的代码组织能力和拆分逻辑能力。

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤1

打开命令提示符，切换到D:\vue\chapter03目录，在该目录下执行如下命令，创建项目。

```
yarn create vite component_basis --template vue
```

步骤2

项目创建完成后，执行如下命令进入项目目录，启动项目。

步骤3

```
cd component_basis
```

步骤4

```
yarn
```

```
yarn dev
```

步骤5

项目启动后，可以使用URL地址<http://127.0.0.1:5173/>进行访问。

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤1

使用VS Code编辑器打开D:\vue\chapter03\component_basis目录。

步骤2

▼ COMPONENT_BASIS

> .vscode

> node_modules

> public

> src

📄 .gitignore

<> index.html

{ } package.json

📖 README.md

JS vite.config.js

👤 yarn.lock

步骤3

步骤4

步骤5

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤1

步骤2

步骤3

步骤4

步骤5

将src\style.css文件中的样式代码全部删除，从而避免项目创建时自带的样式代码影响本案例的实现效果。

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤1

步骤2

步骤3

步骤4

步骤5

创建src\components\OptionsAPI.vue文件，用于演示选项式API的写法，在该文件中实现单击“+1”按钮使数字加1的效果。

```
<template>
  <div>数字: {{ number }}</div>
  <button @click="addNumber">+1</button>
</template>
<script>
export default {
  data() { return { number: 1 } },
  methods: { addNumber() { this.number++ } }
}
</script>
```

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤1

修改src\main.js文件，[切换页面中显示的组件。](#)

步骤2

```
import App from './components/OptionsAPI.vue'
```

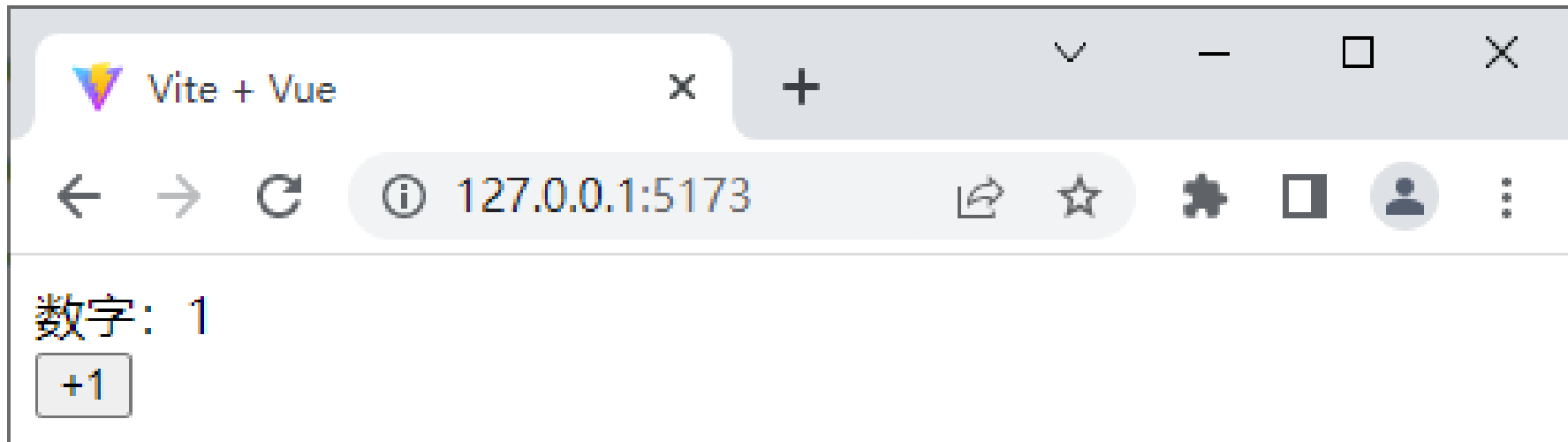
步骤3

步骤4

步骤5

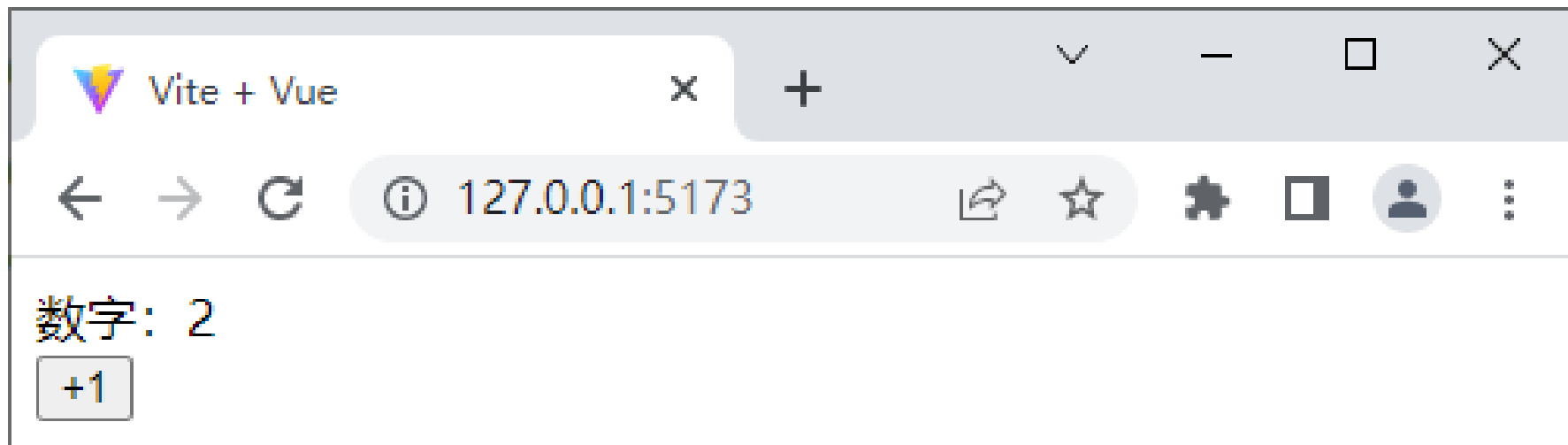
3.1 选项式API和组合式API

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，通过选项式API实现的初始页面效果如下图所示。



3.1 选项式API和组合式API

单击 “+1” 按钮后的页面效果如下图所示。



从上图可以看出，单击 “+1” 按钮后，数字变为2，说明通过选项式API的写法实现数字加1的效果成功。

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤6

创建src\components\CompositionAPI.vue文件，用于演示组合式API的写法，在该文件中实现单击“+1”按钮使数字加1的效果。

步骤7

```
<template>
  <div>数字: {{ number }}</div>
  <button @click="addNumber">+1</button>
</template>
<script setup>
import { ref } from 'vue'
let number = ref(1)
const addNumber = () => { number.value ++ }
</script>
```

3.1 选项式API和组合式API

演示选项式API和组合式API的使用方法

步骤6

修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/CompositionAPI.vue'
```

步骤7

3.1 选项式API和组合式API

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果与通过选项式API实现的初始页面效果相同。

3.2

生命周期函数

3.2 生命周期函数



- 掌握生命周期函数的使用方法，能够灵活运用生命周期函数在特定时间执行特定的操作

3.2 生命周期函数

什么是生命周期函数？



3.2 生命周期函数



在Vue中，组件的**生命周期**是指**每个组件从被创建到被销毁的整个过程**，每个组件都有生命周期。如果想要在**某个特定的时机进行特定的处理**，可以通过**生命周期函数**来完成。随着组件生命周期的变化，**生命周期函数会自动执行**。

3.2 生命周期函数

组合式API下的生命周期函数如下表所示。

函数	说明
onBeforeMount()	组件挂载前
onMounted()	组件挂载成功后
onBeforeUpdate()	组件更新前
onUpdated()	组件中的任意的DOM元素更新后
onBeforeUnmount()	组件实例被销毁前
onUnmounted()	组件实例被销毁后

3.2 生命周期函数

以`onMounted()`函数为例演示生命周期函数的使用。

```
<script setup>
import { onMounted } from 'vue'
onMounted(() => {
  // 执行操作
})
</script>
```

3.2 生命周期函数

演示生命周期函数的使用方法

步骤1

创建src\components\LifecycleHooks.vue文件，用于通过生命周期函数查看在特定时间点下的DOM元素。

步骤2

```
<template> <div class="container">container</div> </template>
<script setup>
import { onBeforeMount, onMounted } from 'vue'
onBeforeMount(() => {
  console.log('DOM元素渲染前', document.querySelector('.container'))
})
onMounted(() => {
  console.log('DOM元素渲染后', document.querySelector('.container'))
})
</script>
```

3.2 生命周期函数

演示生命周期函数的使用方法

步骤1

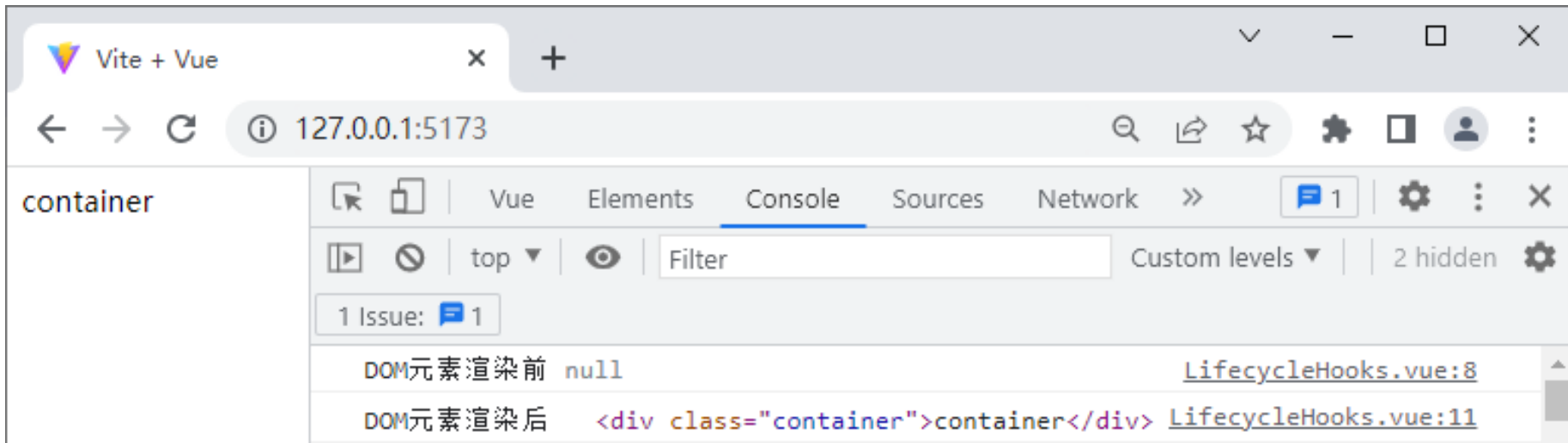
修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

```
import App from './components/LifecycleHooks.vue'
```

步骤2

3.2 生命周期函数

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>并打开控制台，使用生命周期函数的页面效果和控制台如下图所示。



3.2 生命周期函数



多学一招：选项式API下的生命周期函数

选项式API下的生命周期函数如下表所示。

函数	说明
beforeCreate()	实例对象创建前
created()	实例对象创建后
beforeMount()	页面挂载前
mounted()	页面挂载成功后
beforeUpdate()	组件更新前
updated()	组件中的任意的DOM元素更新后
beforeDestroy()	组件实例销毁前
destroyed()	组件实例销毁后

3.2 生命周期函数

演示选项式API下beforeCreate()函数和created()函数的使用。

```
<script>
export default {
  data() { return { value: 'Hello Vue.js' } },
  beforeCreate() {
    console.log('实例对象创建前: ' + this.value)
  },
  created() {
    console.log('实例对象创建后: ' + this.value)
  }
}
</script>
```

3.3

组件的注册和引用

3.3.1 注册组件



- 掌握注册组件的方法，能够运用全局注册或者局部注册的方式完成组件的注册

3.3.1 注册组件

为什么需要注册组件?



3.3.1 注册组件



当在Vue项目中定义了一个新的组件后，要想在其他组件中引用这个新的组件，需要对新的组件进行注册。在注册组件的时候，需要给组件取一个名字，从而区分每个组件，可以采用帕斯卡命名法（PascalCase）为组件命名。

Vue提供了两种注册组件的方式，分别是全局注册和局部注册。

3.3.1 注册组件

1. 全局注册

在实际开发中，如果某个组件的**使用频率很高**，许多组件中都会引用该组件，则推荐将该组件**全局注册**。被全局注册的组件可以在当前Vue项目的任何一个组件内引用。

在Vue项目的src\main.js文件中，通过Vue应用实例的**component()**方法可以**全局注册组件**，该方法的语法格式如下。

```
component('组件名称', 需要被注册的组件)
```

上述语法格式中，**component()**方法接收**两个参数**，第**1**个参数为**组件名称**，注册完成后即可全局使用该组件名称，第**2**个参数为**需要被注册的组件**。

3.3.1 注册组件

在src\main.js文件中注册一个全局组件MyComponent，示例代码如下。

```
import { createApp } from 'vue';
import './style.css'
import App from './App.vue'
import MyComponent from
'./components/MyComponent.vue'
const app = createApp(App)
app.component('MyComponent', MyComponent)
app.mount('#app')
```

3.3.1 注册组件

component()方法支持链式调用，可以连续注册多个组件，示例代码如下。

```
app.component('ComponentA', ComponentA)
    .component('ComponentB', ComponentB)
    .component('ComponentC', ComponentC)
```

3.3.1 注册组件

2. 局部注册

在实际开发中，如果某些组件只在特定的情况下被用到，推荐进行局部注册。局部注册即在某个组件中注册，被局部注册的组件只能在当前注册范围内使用。

局部注册组件的示例代码如下。

```
<script>
import ComponentA from './ComponentA.vue'
export default {
  components: { ComponentA: ComponentA }
}
</script>
```

3.3.1 注册组件

在使用`setup`语法糖时，导入的组件会被自动注册，无须手动注册，导入后可以直接在模板中使用，示例代码如下。

```
<script setup>  
import ComponentA from './ComponentA.vue'  
</script>
```

3.3.2 引用组件



- 掌握引用组件的方法，能够在Vue项目中以标签的形式引用组件

3.3.2 引用组件

将组件注册完成后，若要将组件在页面中渲染出来，需要引用组件。

在组件的<template>标签中可以引用其他组件，被引用的组件需要写成标签的形式，标签名应与组件名对应。组件的标签名可以使用短横线分隔或帕斯卡命名法命名。例如，<my-component>标签和<MyComponent>标签都表示引用MyComponent组件。一个组件可以被引用多次，但不可出现自我引用和互相引用的情况，否则会出现死循环。

3.3.2 引用组件

演示组件的使用方法

步骤1

创建src\components\ComponentUse.vue文件。

步骤2

步骤3

步骤4

步骤5

```
<template>
  <h5>父组件</h5>
  <div class="box">
  </div>
</template>
<style>
.box {
  display: flex;
}
</style>
```

3.3.2 引用组件

演示组件的使用方法

步骤1

修改src\main.js文件，切换页面中显示的组件。

步骤2

```
import App from './components/ComponentUse.vue'
```

步骤3

步骤4

步骤5

3.3.2 引用组件

演示组件的使用方法

步骤1

步骤2

步骤3

步骤4

步骤5

创建src\components\GlobalComponent.vue文件，表示全局组件。

```
<template>
  <div class="global-container"> <h5>全局组件</h5> </div>
</template>
<style>
.global-container {
  border: 1px solid black;
  height: 50px;
  flex: 1;
}
</style>
```

3.3.2 引用组件

演示组件的使用方法

创建src\components\LocalComponent.vue文件，表示局部组件。

步骤1

步骤2

步骤3

步骤4

步骤5

```
<template>
  <div class="local-container">
    <h5>局部组件</h5>
  </div>
</template>
<style>
.local-container {
  border: 1px dashed black;
  height: 50px;
  flex: 1;
}
</style>
```

3.3.2 引用组件

演示组件的使用方法

步骤1

修改src\main.js文件，导入GlobalComponent组件并调用component()方法全局注册GlobalComponent组件。

步骤2

步骤3

步骤4

步骤5

```
import { createApp } from 'vue'
import './style.css'
import App from './components/ComponentUse.vue'
import GlobalComponent from
'./components/GlobalComponent.vue'
const app = createApp(App)
app.component('GlobalComponent', GlobalComponent)
app.mount('#app')
```

3.3.2 引用组件

演示组件的使用方法

步骤6

修改src\components\ComponentUse.vue文件，添加代码导入LocalComponent组件。

```
<script setup>
import LocalComponent from './LocalComponent.vue'
</script>
```

步骤7

3.3.2 引用组件

演示组件的使用方法

步骤6

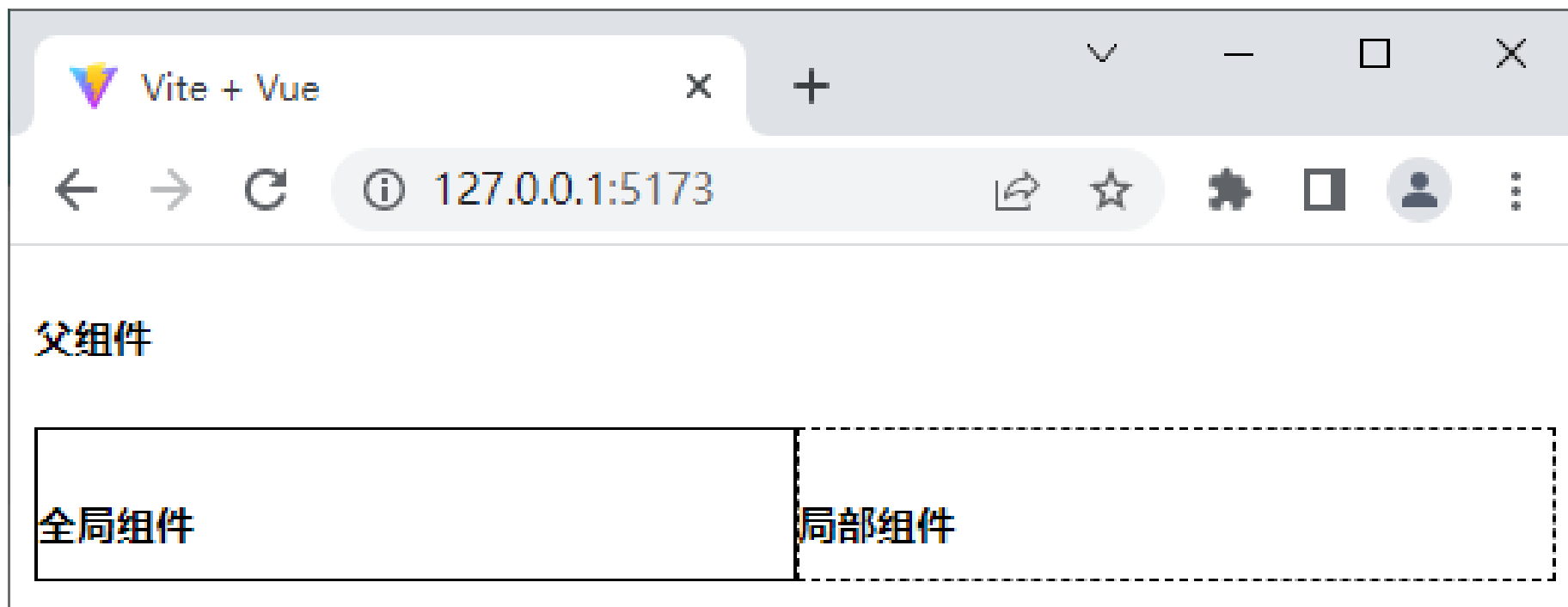
修改src\components\ComponentUse.vue文件，在class为box的<div>标签中引用GlobalComponent组件和LocalComponent组件。

```
<div class="box">  
  <GlobalComponent />  
  <LocalComponent />  
</div>
```

步骤7

3.3.2 引用组件

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，引用组件后的页面效果如下图所示。



3.4

解决组件之间的样式冲突

3.4 解决组件之间的样式冲突



- 掌握组件之间样式冲突问题的解决方法，能够运用 `<style>` 标签的 `scoped` 属性和深度选择器解决组件之间样式冲突的问题

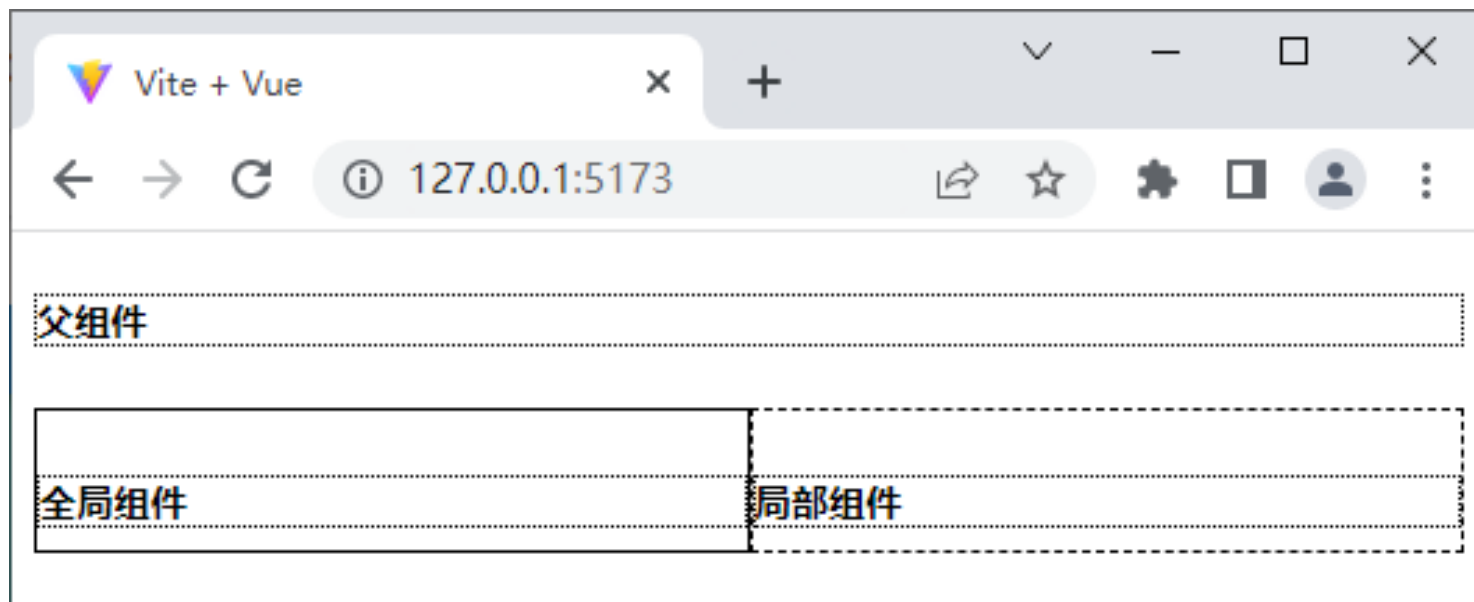
3.4 解决组件之间的样式冲突

在默认情况下，写在Vue组件中的样式会全局生效，很容易造成多个组件之间的样式冲突问题。例如，为ComponentUse组件中的h5元素添加边框样式，具体代码如下。

```
h5 {  
  border: 1px dotted black;  
}
```

3.4 解决组件之间的样式冲突

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，添加边框样式后的页面效果如下图所示。



从上图可以看出，[ComponentUse组件](#)、[GlobalComponent组件](#)和[LocalComponent组件](#)中h5元素的[边框样式都发生了改变](#)，但是代码中只有ComponentUse组件设置了边框样式效果，说明组件之间存在样式冲突。

3.4 解决组件之间的样式冲突

为什么组件之间会产生
样式冲突?



3.4 解决组件之间的样式冲突



导致组件之间样式冲突的根本原因是：在单页Web应用中，所有组件的DOM结构都是基于唯一的index.html页面进行呈现的。每个组件中的样式都可以影响整个页面中的DOM元素。

在Vue中可以使用scoped属性和深度选择器来解决组件之间的样式冲突。

3.4 解决组件之间的样式冲突

1. scoped属性

Vue为<style>标签提供了scoped属性，用于解决组件之间的样式冲突。

为<style>标签添加scoped属性后，Vue会自动为当前组件的DOM元素添加一个唯一的自定义属性（如data-v-7ba5bd90），并在样式中为选择器添加自定义属性（如.list[data-v-7ba5bd90]），从而限制样式的作用范围，防止组件之间的样式冲突问题。

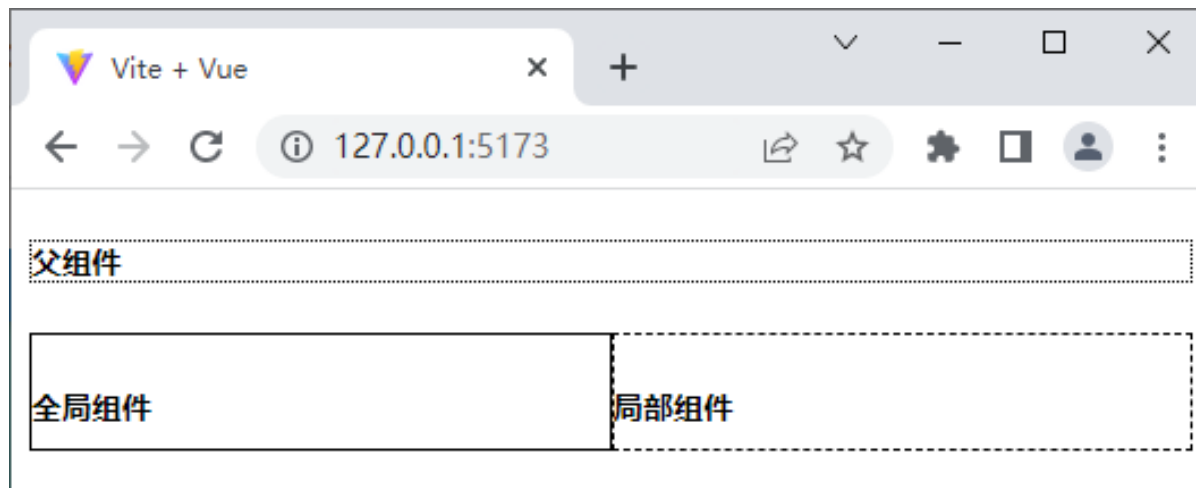
3.4 解决组件之间的样式冲突

下面演示scoped属性的使用。

修改ComponentUse组件，为<style>标签添加scoped属性，具体代码如下。

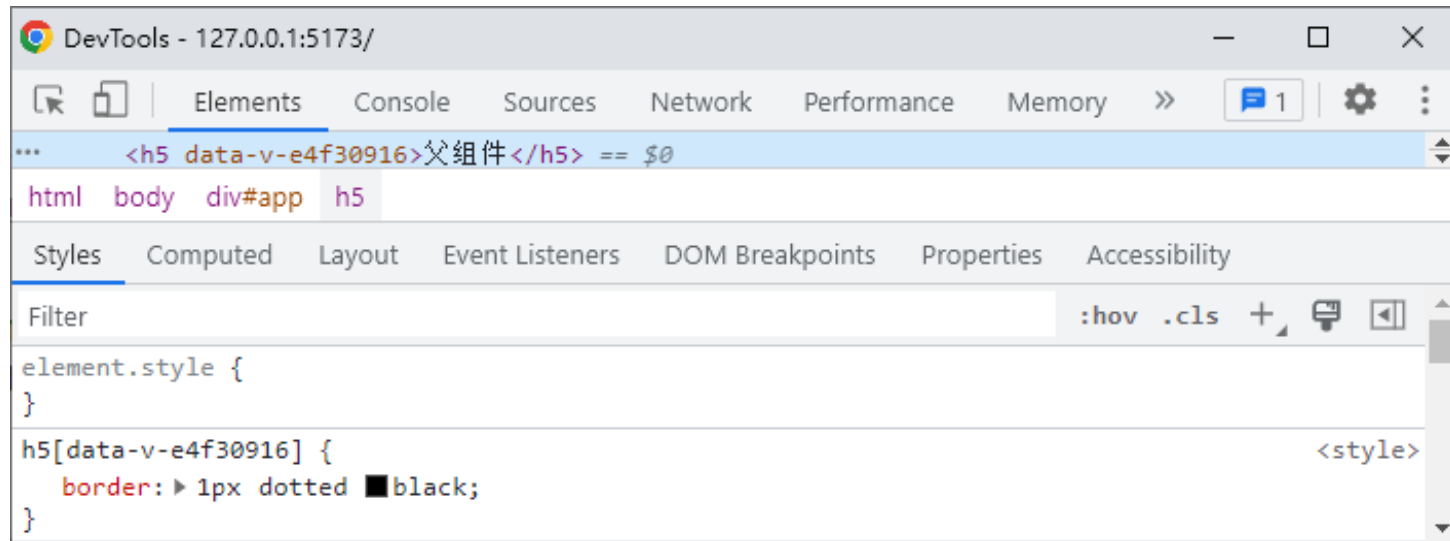
```
<style scoped>
```

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，在<style>标签中添加scoped属性的页面效果如下图所示。



3.4 解决组件之间的样式冲突

打开开发者工具，切换到Elements面板，查看父组件的h5元素的代码，如下图所示。



从上图可以看出，当`<style>`标签添加`scoped`属性后，`h5`元素和相应的选择器被Vue自动添加了`data-v-e4f30916`属性，从而解决了样式冲突的问题。

3.4 解决组件之间的样式冲突

2. 深度选择器

如果给当前组件的<style>标签添加了scoped属性，则当前组件的样式对其子组件是不生效的。

如果在[添加了scoped属性后](#)还需要[让某些样式对子组件生效](#)，则可以使用[深度选择器](#)来实现。

深度选择器通过[:deep\(\)伪类](#)来实现，在其小括号中可以定义[用于子组件的选择器](#)，例如，
“:deep(.title)” 被编译之后生成选择器的格式为 “[data-v-7ba5bd90] .title” 。

3.4 解决组件之间的样式冲突

演示如何通过ComponentUse组件更改 LocalComponent组件的样式

步骤1

为LocalComponent组件的h5元素添加class属性。

```
<h5 class="title">局部组件</h5>
```

步骤2

3.4 解决组件之间的样式冲突

演示如何通过ComponentUse组件更改 LocalComponent组件的样式

步骤1

在ComponentUse组件中定义.title的样式。

```
:deep(.title){  
  border: 3px dotted black;  
}
```

步骤2

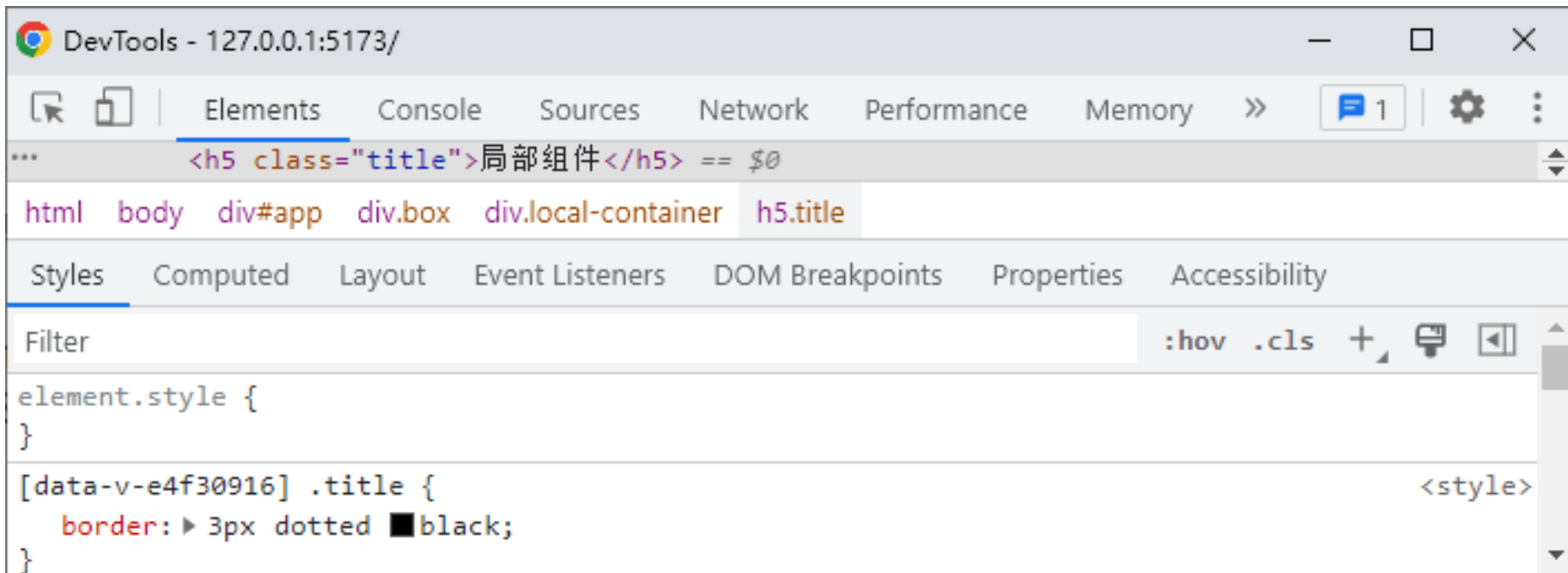
3.4 解决组件之间的样式冲突

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，添加深度选择器实现样式穿透的页面效果如下图所示。



3.4 解决组件之间的样式冲突

打开开发者工具，切换到Elements面板，查看LocalComponent组件的h5元素的代码，页面效果如下图所示。



3.5

父组件向子组件传递数据

3.5.1 声明props



- 掌握声明props, 能够在子组件中声明props

3.5.1 声明props



若想实现父组件向子组件传递数据，需要先在子组件中声明props，表示子组件可以从父组件中接收哪些数据。

3.5.1 声明props

在不使用setup语法糖的情况下，可以使用props选项声明props。props选项的形式可以是对象或字符串数组。声明对象形式的props的语法格式如下。

```
<script>
export default {
  props: {
    自定义属性A: 类型,
    自定义属性B: 类型,
    .....
  }
}
</script>
```

如果不需要限制props的类型，可以声明字符串数组形式的props，示例代码如下。

```
props: ['自定义属性A', '自定义属性B'],
```


3.5.1 声明props

当使用setup语法糖时，可使用defineProps()函数声明props，语法格式如下。

```
<script setup>  
const props = defineProps({'自定义属性A': 类型}, {'自定义属性B': 类型})  
</script>
```

使用defineProps()函数声明字符串数组形式的props，语法格式如下。

```
const props = defineProps(['自定义属性A', '自定义属性B'])
```

3.5.1 声明props

在组件中声明了props后，可以直接在模板中输出每个prop的值，语法格式如下。

```
<template>
  {{ 自定义属性A }}
  {{ 自定义属性B }}
</template>
```

3.5.2 静态绑定props



- 掌握父组件向子组件传递静态数据的方法，能够通过静态绑定props实现数据传递

3.5.2 静态绑定props



当在父组件中引用了子组件后，如果子组件中声明了props，则可以在父组件中向子组件传递数据。如果传递的数据是固定不变的，则可以通过静态绑定props的方式为子组件传递数据。

3.5.2 静态绑定props

通过静态绑定props的方式为子组件传递数据，其语法格式如下。

```
<子组件标签名 自定义属性A="数据" 自定义属性B="数据" />
```

在上述语法格式中，父组件向子组件的props传递了静态的数据，属性值默认为字符串类型。

注意

如果子组件中未声明props，则父组件向子组件中传递的数据会被忽略，无法被子组件使用。

3.5.2 静态绑定props

演示父组件向子组件传递数据的方法

创建src\components\Count.vue文件，用于展示子组件的相关内容。

步骤1

```
<template>
  初始值为: {{ num }}
</template>
<script setup>
const props = defineProps({
  num: String
})
</script>
```

步骤2

步骤3

3.5.2 静态绑定props

演示父组件向子组件传递数据的方法

步骤1

创建src\components\Props.vue文件，用于展示父组件的相关内容。

步骤2

```
<template>
  <Count num="1" />
</template>
<script setup>
import Count from './Count.vue'
</script>
```

步骤3

3.5.2 静态绑定props

演示父组件向子组件传递数据的方法

步骤1

修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

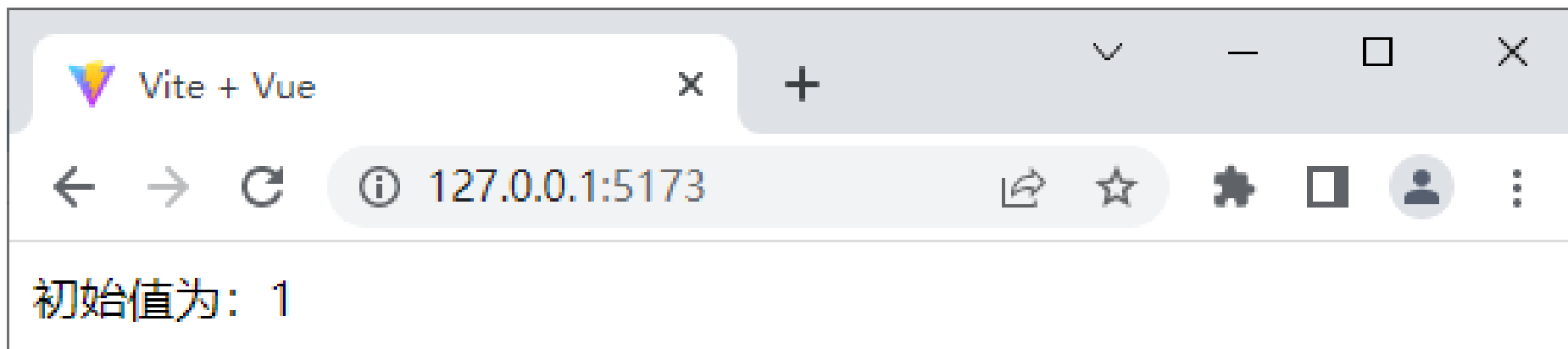
```
import App from './components/Props.vue'
```

步骤2

步骤3

3.5.2 静态绑定props

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，父组件向子组件中传递数据的页面效果如下图所示。



3.5.3 动态绑定props



- 掌握父组件向子组件传递动态数据的方法，能够使用props实现数据传递

3.5.3 动态绑定props



在父组件中使用v-bind可以为子组件动态绑定props，任意类型的值都可以传给子组件的props，包括字符串、数字、布尔值、数组、对象等。

3.5.3 动态绑定props

1. 字符串

从父组件中为子组件传递字符串类型的props数据，示例代码如下。

```
<template>
  <Child :init="username" />
</template>
<script setup>
import Child from './Child.vue'
import { ref } from 'vue'
const username = ref('小圆')
</script>
```

3.5.3 动态绑定props

上述代码用到了名称为Child的子组件，该子组件的示例代码如下。

```
<template> </template>
<script setup>
const props = defineProps(['init'])
console.log(props)
</script>
```

3.5.3 动态绑定props

2. 数字

从父组件中为子组件传递数字类型的props数据，示例代码如下。

```
<template>
  <Child :init="12" />
  <Child :init="age" />
</template>
<script setup>
import Child from './Child.vue'
import { ref } from 'vue'
const age = ref(12)
</script>
```

3.5.3 动态绑定props

3. 布尔值

从父组件中为子组件传递布尔类型的props数据，示例代码如下。

```
<template>
  <Child init />
  <Child :init="false" />
  <Child :init="isFlag" />
</template>
<script setup>
import Child from './Child.vue'
import { ref } from 'vue'
const isFlag = ref(true)
</script>
```

3.5.3 动态绑定props

4. 数组

从父组件中为子组件传递数组类型的props数据，示例代码如下。

```
<template>
  <Child :init="['唱歌', '跳舞', '滑冰']" />
  <Child :init="hobby" />
</template>
<script setup>
import Child from './Child.vue'
import { ref } from 'vue'
const hobby = ref(['唱歌', '跳舞', '滑冰'])
</script>
```


3.5.3 动态绑定props

5. 对象

从父组件中为子组件传入一个对象类型的props数据，或者将对象中的部分属性作为被传入的props数据，示例代码如下。

```
<template>
  <Child :init= "{ height: '180厘米' , weight: '70千克' }" />
  <Child :height="bodyInfo.height" :weight="bodyInfo.weight" />
  <Child v-bind="bodyInfo" />
</template>
```

3.5.3 动态绑定props

>> 接上页代码

```
<script setup>
import Child from './Child.vue'
import { reactive } from 'vue'
const bodyInfo = reactive({
  height: '180厘米',
  weight: '70千克'
})
</script>
```

3.5.3 动态绑定props



props单向数据流



在Vue中，所有的props都遵循单向数据流原则，props数据因父组件的更新而变化，变化后的数据将向下流往子组件，而且不会逆向传递，这样可以防止因子组件意外变更props导致数据流向难以理解的问题。

每次父组件绑定的props发生变更时，子组件中的props都将会刷新为最新的值。开发者不应该在子组件内部改变props，如果这样做，Vue会在浏览器的控制台中发出警告。

3.5.4 验证props



- 掌握验证props的方法，能够对从父组件传递过来的props数据进行合法性校验

3.5.4 验证props



在封装组件时，可以在子组件中对从父组件传递过来的props数据进行合法性校验，从而防止出现数据不合法的问题。

3.5.4 验证props

使用字符串数组形式的props的缺点是无法为每个prop指定具体的数据类型，而使用对象形式的props的优点是可以对每个prop进行数据类型的校验。

对象形式的props可以使用多种验证方案，包括基础类型检查、必填项的校验、属性默认值、自定义验证函数等。在声明props时，可以添加验证方案。

3.5.4 验证props

1. 基础类型检查

在开发中，有时需要对从父组件中传递过来的props数据进行基础类型检查，这时可以通过type属性检查合法的类型，如果从父组件中传递过来的值不符合此类型，则会报错。

常见的类型有String（字符串）、Number（数字）、Boolean（布尔值）、Array（数组）、Object（对象）、Date（日期）、Function（函数）、Symbol（符号）以及任何自定义构造函数。

3.5.4 验证props

为props指定基础类型检查，示例代码如下。

```
props: {  
  自定义属性A: String,           // 字符串  
  自定义属性B: Number,          // 数字  
  自定义属性C: Boolean,         // 布尔值  
  自定义属性D: Array,           // 数组  
  自定义属性E: Object,          // 对象  
  自定义属性F: Date,            // 日期  
  自定义属性G: Function,        // 函数  
  自定义属性H: Symbol,          // 符号  
}
```


3.5.4 验证props

通过配置对象的形式定义验证规则，示例代码如下。

```
props: {  
  自定义属性: { type: Number },  
}
```

如果某个prop的类型不唯一，可以通过数组的形式为其指定多个可能的类型，示例代码如下。

```
props: {  
  自定义属性: { type: [String, Array] }, // 字符串或数组  
}
```

3.5.4 验证props

2. 必填项的校验

父组件向子组件传递props数据时，有可能传递的数据为空，但是在子组件中要求该数据是必须传递的。此时，可以在声明props时通过required属性设置必填项，强调组件的使用者必须传递属性的值，示例代码如下。

```
props: {  
  自定义属性: { required: true },  
}
```

3.5.4 验证props

3. 属性默认值

在声明props时，可以通过default属性定义属性默认值，当父组件没有向子组件的属性传递数据时，属性将会使用默认值，示例代码如下。

```
props: {  
  自定义属性: { default: 0 },  
}
```

3.5.4 验证props

4. 自定义验证函数

如果需要对从父组件中传入的数据进行验证，可以通过`validator()`函数来实现。`validator()`函数可以将`prop`的值作为唯一参数传入自定义验证函数，如果验证失败，则会在控制台中发出警告。为`prop`属性指定自定义验证函数的示例代码如下。

```
props: {  
  自定义属性: {  
    validator(value) {  
      return ['success', 'warning', 'danger'].indexOf(value) !== -1;  
    },  
  },  
}
```

3.6

子组件向父组件传递数据

3.6.1 在子组件中声明自定义事件



- 掌握在子组件中声明自定义事件的方法，能够在子组件中声明自定义事件

3.6.1 在子组件中声明自定义事件

若想使用自定义事件，首先需要在子组件中声明自定义事件。

在不使用setup语法糖时，可以通过emits选项声明自定义事件，示例代码如下。

```
<script>
export default {
  emits: ['demo']
}
</script>
```

3.6.1 在子组件中声明自定义事件

在使用setup语法糖时，需要通过调用`defineEmits()`函数声明自定义事件，示例代码如下。

```
<script setup>  
const emit = defineEmits(['demo'])  
</script>
```

在上述代码中，`defineEmits()`函数的参数与`emits`选项中的相同。

3.6.2 在子组件中触发自定义事件



- 掌握在子组件中触发自定义事件的方法，能够在子组件中触发自定义事件

3.6.2 在子组件中触发自定义事件

在子组件中[声明自定义事件](#)后，接着需要在[子组件中触发自定义事件](#)。

当使用场景简单时，可以使用[内联事件处理器](#)，通过调用[\\$emit\(\)](#)方法触发自定义事件，将数据传递给使用的组件，示例代码如下。

```
<button @click="$emit('demo', 1)">按钮</button>
```

在上述代码中，[\\$emit\(\)](#)方法的第1个参数为[字符串类型](#)的[自定义事件的名称](#)，第2个参数为[需要传递的数据](#)，当触发当前组件的事件时，该数据会传递给父组件。

3.6.2 在子组件中触发自定义事件

除了使用内联方式外，还可以[直接定义方法来触发自定义事件](#)。

在不使用setup语法糖时，可以从setup()函数的第2个参数（即[setup上下文对象](#)）来访问到[emit\(\)方法](#)，示例代码如下。

```
export default {  
  setup(props, ctx) {  
    const update = () => {  
      ctx.emit('demo', 2)  
    }  
    return { update }  
  }  
}
```

3.6.2 在子组件中触发自定义事件

如果使用setup语法糖，可以调用`emit()`函数来实现，示例代码如下。

```
<script setup>
const update = () => {
  emit('demo', 2)
}
</script>
```

3.6.3 在父组件中监听自定义事件



- 掌握在父组件中监听自定义事件的方法，能够在父组件中监听自定义事件

3.6.3 在父组件中监听自定义事件

在父组件中通过v-on可以[监听子组件中抛出的事件](#)，示例代码如下。

```
<子组件名 @demo="fun" />
```

在上述代码中，当触发demo事件时，会接收到从子组件中传递的参数，同时会执行fun()方法。[父组件](#)可以通过value属性接收从子组件中传递来的参数。

在[父组件](#)中定义fun()方法，示例代码如下。

```
const fun = value => {  
  console.log(value)  
}
```

3.6.3 在父组件中监听自定义事件

演示子组件向父组件传递数据的方法

步骤1

创建src\components\CustomSubComponent.vue文件，用于展示子组件的相关内容。

步骤2

步骤3

```
<template>
  <p>count值为: {{ count }}</p>
  <button @click="add">加n</button>
</template>
<script setup>
import { ref } from 'vue'
const emit = defineEmits(['updateCount'])
const count = ref(1)
const add = () => { count.value++
  emit('updateCount', 2) }
</script>
```

3.6.3 在父组件中监听自定义事件

演示子组件向父组件传递数据的方法

步骤1

创建src\components\CustomEvents.vue文件，用于展示父组件的相关内容。

步骤2

```
<template>
  <p>父组件当前的值为: {{ number }}</p>
  <CustomSubComponent @updateCount="updateEmitCount" />
</template>
<script setup>
import CustomSubComponent from './CustomSubComponent.vue'
import { ref } from 'vue'
const number = ref(10)
const updateEmitCount = (value) => { number.value += value }
</script>
```

步骤3

3.6.3 在父组件中监听自定义事件

演示子组件向父组件传递数据的方法

步骤1

修改src\main.js文件，[切换](#)页面中[显示](#)的[组件](#)。

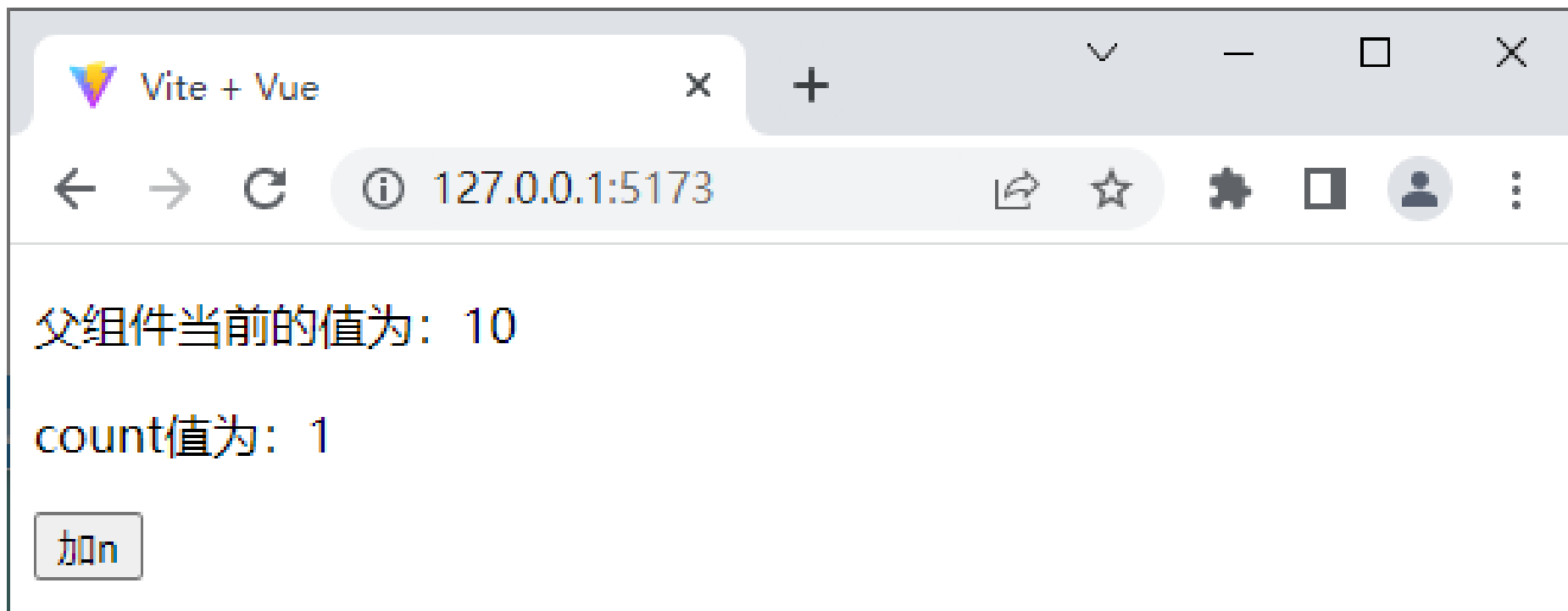
```
import App from './components/CustomEvents.vue'
```

步骤2

步骤3

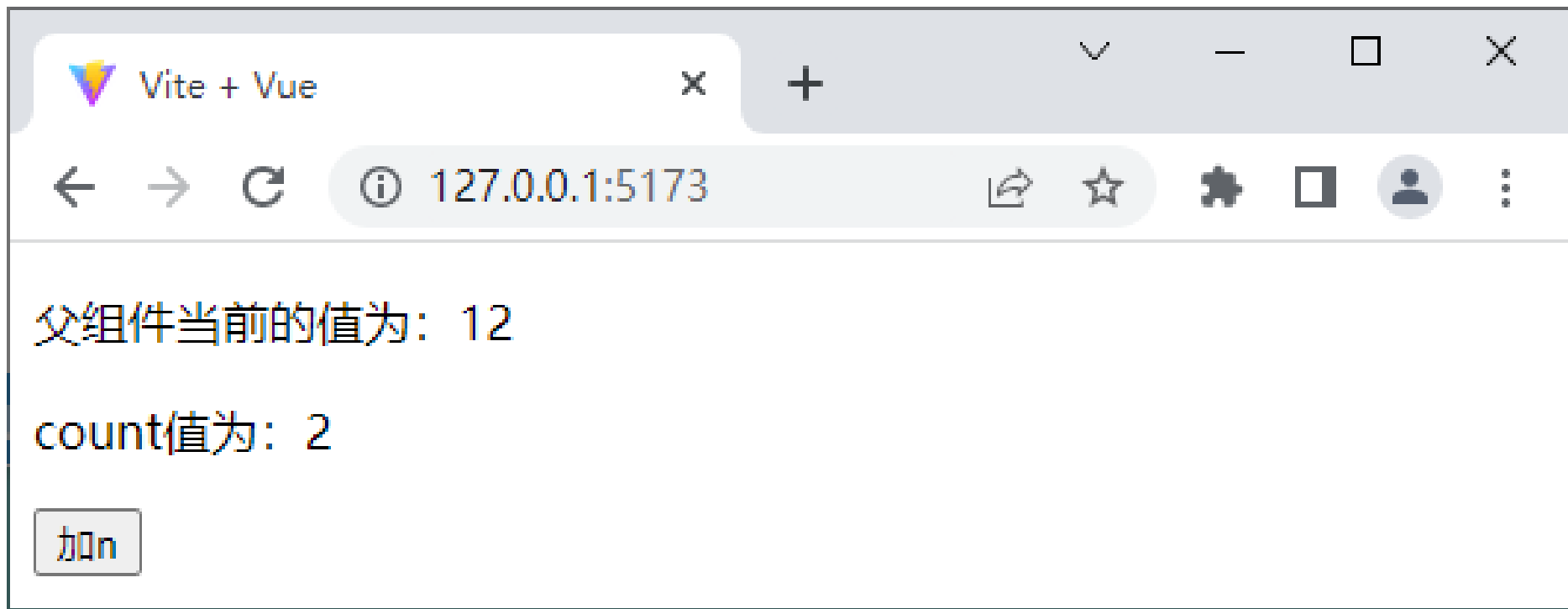
3.6.3 在父组件中监听自定义事件

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下图所示。



3.6.3 在父组件中监听自定义事件

单击“加n”按钮后的页面效果如下图所示。



3.7

跨级组件之间的数据传递

3.7 跨级组件之间的数据传递



- 掌握跨级组件之间的传递数据的方法，
能够使用依赖注入实现数据共享

3.7 跨级组件之间的数据传递

如何实现跨级组件之间的
数据传递？



3.7 跨级组件之间的数据传递



Vue提供了跨级组件之间数据传递的方式——**依赖注入**。一个**父组件**相对于其所有的后代组件而言，可作为**依赖提供者**。而任何后代的组件树，无论层级多深，都可以注入由父组件提供的依赖。

对于**父组件**而言，如果要**为后代组件提供数据**，需要使用**provide()**函数。对于**子组件**而言，如果想要**注入上层组件或整个应用提供的数据**，需要使用**inject()**函数。

3.7 跨级组件之间的数据传递

1. provide()函数

provide()函数可以提供一个值，用于被后代组件注入。provide()函数的语法格式如下。

```
provide(注入名, 注入值)
```

provide()函数可以接收2个参数，第1个参数是注入名，后代组件会通过注入名查找所需的注入值；第2个参数是注入值，值可以是任意类型，包括响应式数据，例如通过ref()函数创建的数据。

3.7 跨级组件之间的数据传递

在不使用setup语法糖的情况下，`provide()`函数必须在组件的`setup()`函数中调用。使用`provide()`函数的示例代码如下。

```
<script>
import { ref, provide } from 'vue'
export default {
  setup() {
    const count = ref(1)
    provide( 'message', count )
  }
}
</script>
```

3.7 跨级组件之间的数据传递

当使用[setup语法糖](#)时，使用provide()函数的示例代码如下。

```
<script setup>
import { provide } from 'vue'
provide('message', 'Hello Vue.js')
</script>
```

provide()函数除了可以在某个组件中提供依赖外，还可以[全局提供依赖](#)。例如，在src/main.js文件中添加全局依赖，示例代码如下

```
const app = createApp(App)
app.provide('message', 'Hello Vue.js')
```

3.7 跨级组件之间的数据传递

2. inject()函数

通过`inject()`函数可以注入上层组件或者整个应用提供的数据。`inject()`函数的语法格式如下。

```
inject(注入值, 默认值, 布尔值)
```

`inject()`函数有3个参数。

- 第1个参数是注入值，Vue会遍历父组件，通过匹配注入的值来确定所提供的值，如果父组件链上多个组件为同一个数据提供了值，那么距离更近的组件将会覆盖更远的组件所提供的值。

3.7 跨级组件之间的数据传递

- 第2个参数是可选的，用于在**没有匹配到注入的值时使用默认值**。第2个参数可以是工厂函数，用于返回某些创建起来比较复杂的值。如果提供的默认值是函数，还需要将false作为第3个参数传入，表明这个函数就是默认值，而不是工厂函数。
- 第3个参数是可选的，类型为**布尔值**，当参数值为false时，表示默认值是函数；当参数值为true时，表示默认值为工厂函数；当省略参数值时，表示默认值为其他类型的数据，不是函数或工厂函数。

3.7 跨级组件之间的数据传递

在不使用setup语法糖的情况下，`inject()`函数必须在组件的`setup()`函数中调用。使用`inject()`函数的示例代码如下。

```
<script>
import { inject } from 'vue';
export default {
  setup() {
    const count = inject('count')
    const foo = inject('foo', 'default value')
    const baz = inject('foo', () => new Map())
    const fn = inject('function', () => { }, false)
  }
}
</script>
```

3.7 跨级组件之间的数据传递

当使用setup语法糖时，使用inject()函数的示例代码如下。

```
<script setup>  
import { inject } from 'vue';  
const count = inject('count')  
</script>
```

3.7 跨级组件之间的数据传递

演示跨级组件之间的数据传递

步骤1

创建src\components\ProvideChildren.vue文件，用于展示子组件中的相关内容。

```
<template>
  <div>子组件</div>
</template>
```

步骤2

步骤3

步骤4

步骤5

3.7 跨级组件之间的数据传递

演示跨级组件之间的数据传递

步骤1

创建src\components\ProvideParent.vue文件，用于展示父组件中的相关内容。

步骤2

```
<template>
  <div>父组件</div>
  <hr>
  <ProvideChildren />
</template>
<script setup>
import ProvideChildren from './ProvideChildren.vue'
</script>
```

步骤3

步骤4

步骤5

3.7 跨级组件之间的数据传递

演示跨级组件之间的数据传递

步骤1

创建src\components\ProvideGrand.vue文件，用于展示父组件的父组件（即爷爷组件）中的相关内容

步骤2

步骤3

```
<template> <div>爷爷组件</div> <hr> <ProvideParent />
</template>
<script setup>
import ProvideParent from './ProvideParent.vue'
import { ref, provide } from 'vue'
let money = ref(1000)
let updateMoney = (value) => { money.value += value}
provide('money', money)
provide('updateMoney', updateMoney)
</script>
```

步骤4

步骤5

3.7 跨级组件之间的数据传递

演示跨级组件之间的数据传递

步骤1

步骤2

步骤3

步骤4

步骤5

修改src\components\ProvideChildren.vue文件，通过inject()函数接收爷爷组件中传过来的数据。

```
<template>
  <div>子组件</div> <hr> 从爷爷组件接收到的资金:
  {{ money }}
  <button @click="updateMoney(500)">单击按钮增加资金
</button>
</template>
<script setup>
import { inject } from 'vue'
let money = inject('money')
let updateMoney = inject('updateMoney')
</script>
```

3.7 跨级组件之间的数据传递

演示跨级组件之间的数据传递

步骤1

修改src\main.js文件，[切换页面中显示的组件。](#)

步骤2

```
import App from './components/ProvideGrand.vue'
```

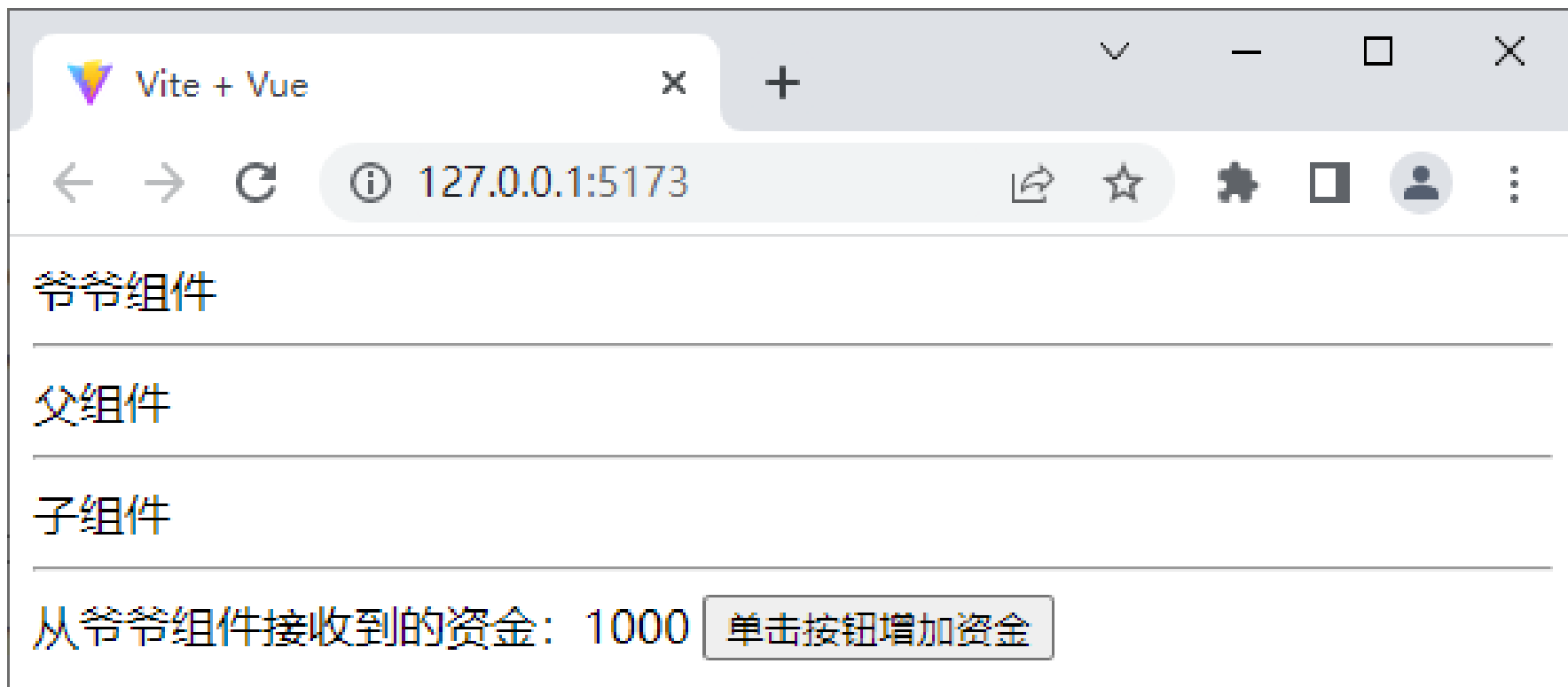
步骤3

步骤4

步骤5

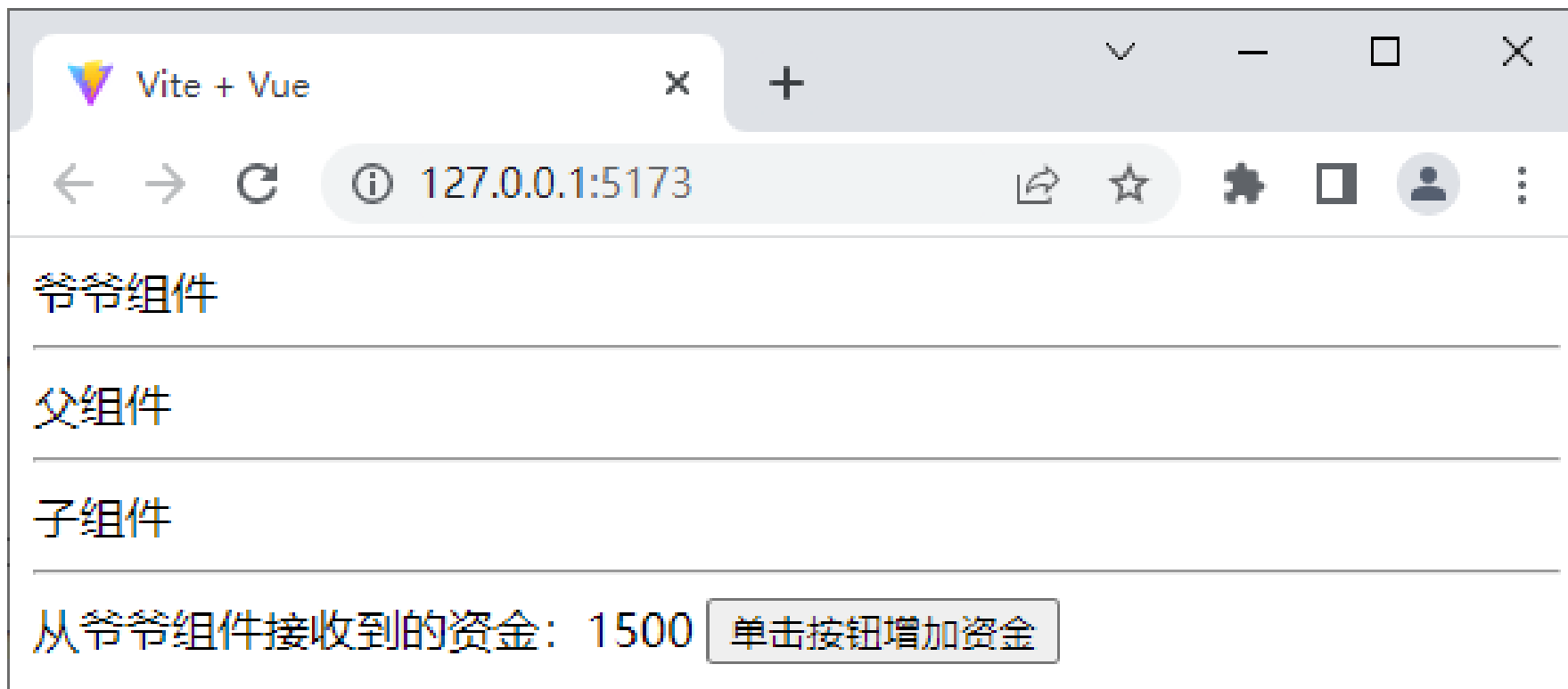
3.7 跨级组件之间的数据传递

保存上述代码后，在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下图所示。



3.7 跨级组件之间的数据传递

单击“单击按钮增加资金”按钮后页面效果如下图所示。



3.8

阶段案例——待办事项

3.8 阶段案例——待办事项



- 掌握待办事项，能够实现待办事项的开发

3.8 阶段案例——待办事项

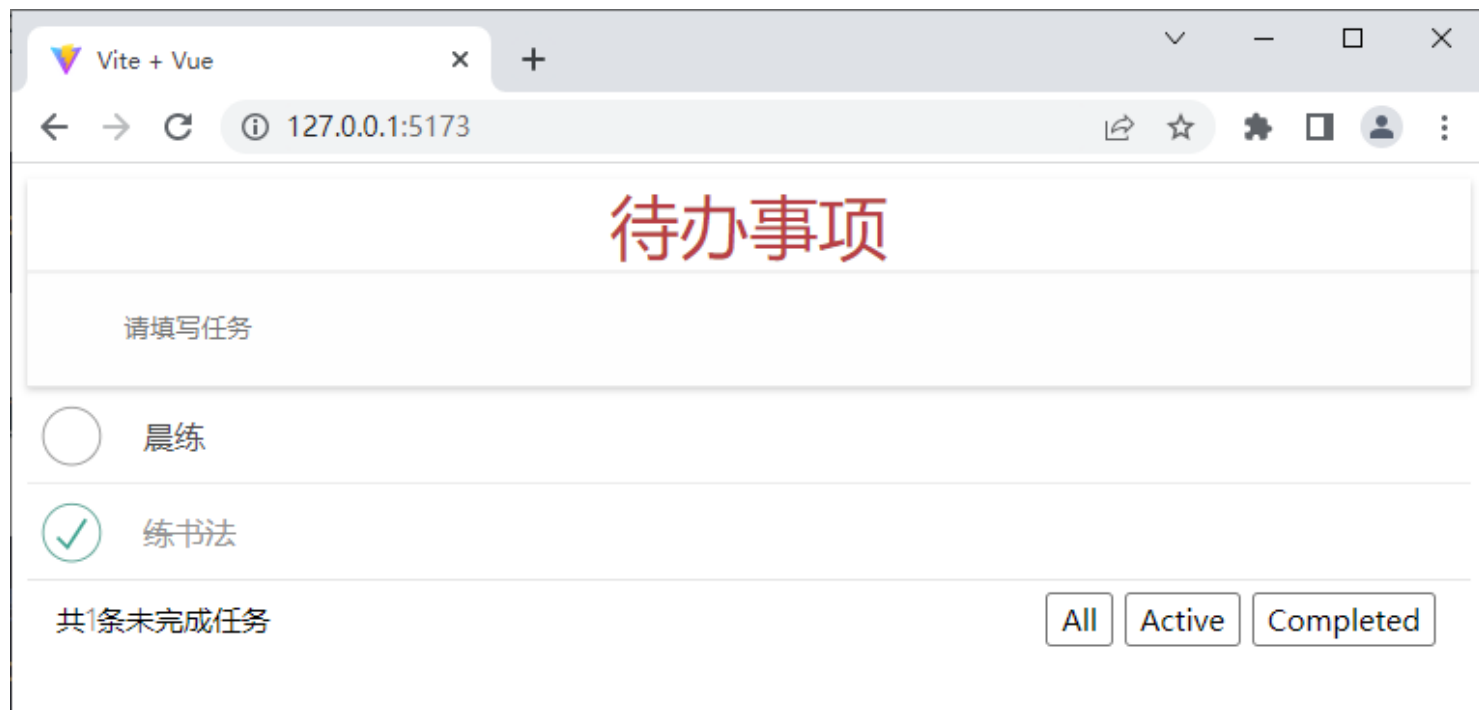


在日常生活中，人们通常倾向于对生活和工作进行提前规划，这样可以更合理地对时间进行划分，从而提高效率。本节将通过讲解**待办事项**案例，使读者巩固所学的Vue中组件的使用、数据共享等知识点。

3.8 阶段案例——待办事项

1. 初始页面效果

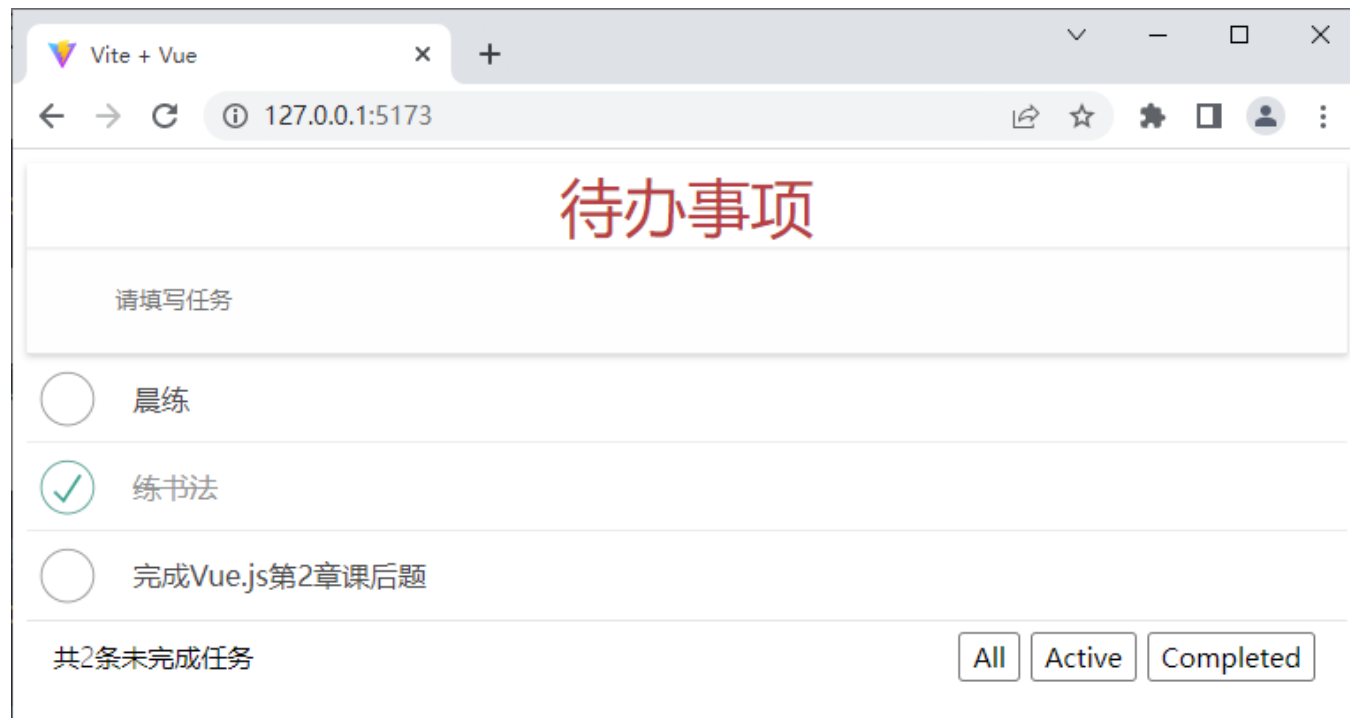
“待办事项”案例的页面结构分为上、中、下3个部分，上半部分为输入区域，中部为列表区域，下半部分为任务状态区域。



3.8 阶段案例——待办事项

2. 新增任务

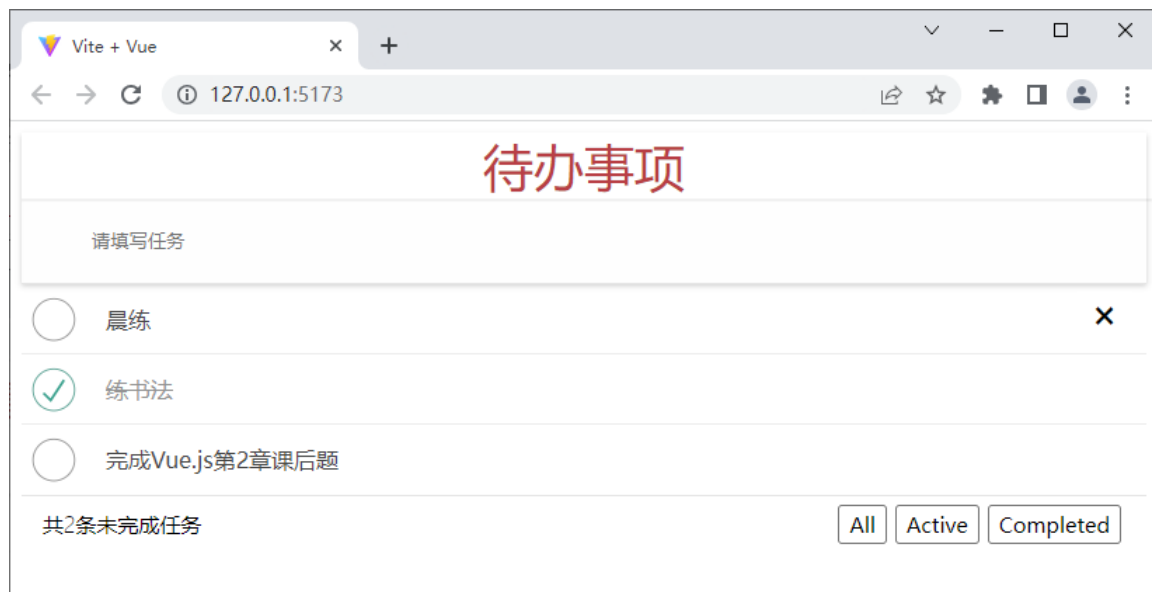
在文本框中输入内容后，在键盘上按下回车键，即可[添加任务到待办列表中](#)，添加“完成Vue.js第2章课后题”任务后的页面效果如下图所示。



3.8 阶段案例——待办事项

3. 删除任务

当鼠标指针移入列表区域中“晨练”任务时，页面效果如下图所示。



单击“晨练”任务右侧的“x”按钮，即可删除该任务。

3.8 阶段案例——待办事项

4. 切换任务状态

在本案例中，任务状态分为全部任务、待办任务、已办任务3种。单击待办任务前的复选框，即可将待办任务状态改为已完成，并将该任务添加到已办任务中。

5. 展示任务数的条数

在任务状态区域左侧会展示任务的总条数。

3.8 阶段案例——待办事项

6. 切换任务列表

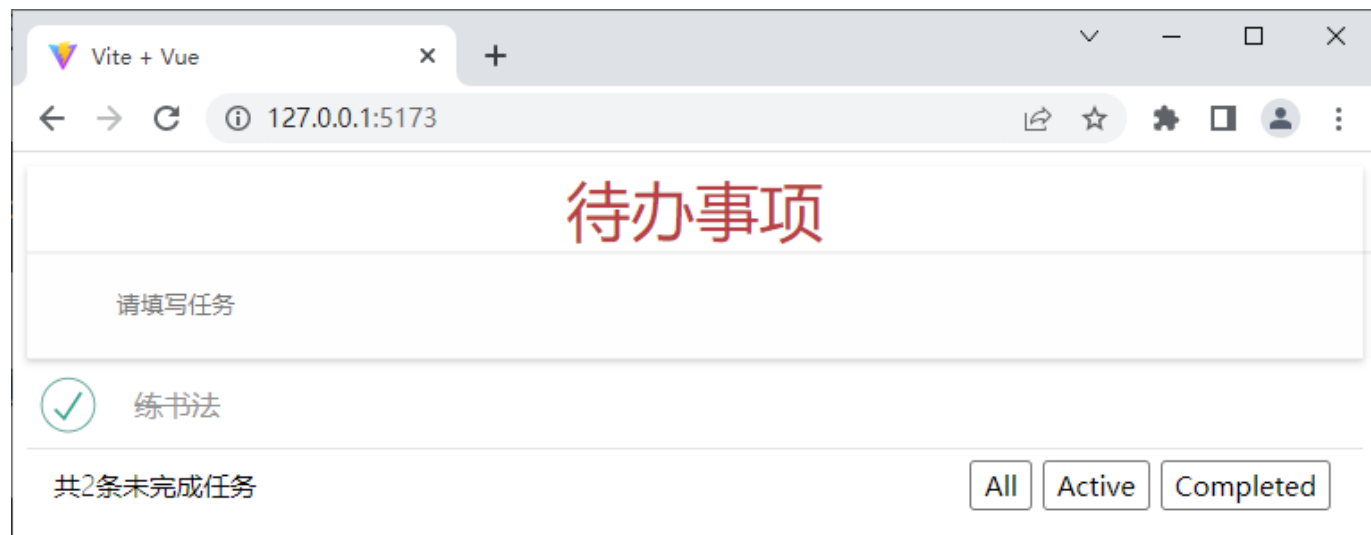
单击任务状态区域的“**All**” “**Active**” “**Completed**” 分别会切换到**全部任务**、**待办任务**、**已办任务列表**。默认情况下，任务状态区域会展示全部任务。
待办任务列表如下图所示。



3.8 阶段案例——待办事项

7. 切换任务列表

已办任务列表如下图所示。



本章小结

本章主要讲解了Vue中组件的基础知识，内容主要包括选项式API和组合式API、生命周期函数、组件的注册和引用、解决组件之间的样式冲突、父组件向子组件传递数据、子组件向父组件传递数据和跨级组件之间的数据传递，最后通过“待办事项”案例的开发，对组件知识进行了综合运用。通过本章的学习，读者能够对Vue的组件有整体的认识，能够利用组件进行项目开发。